Evolutionary Computation

PRESENTED BY

Al-Hutheily 胡天 1820222022

Aimal K. 麦默德 1820222024

Ahmad Wakil 巴戈 1820222043

Tilan W. 罗卡 1820222033

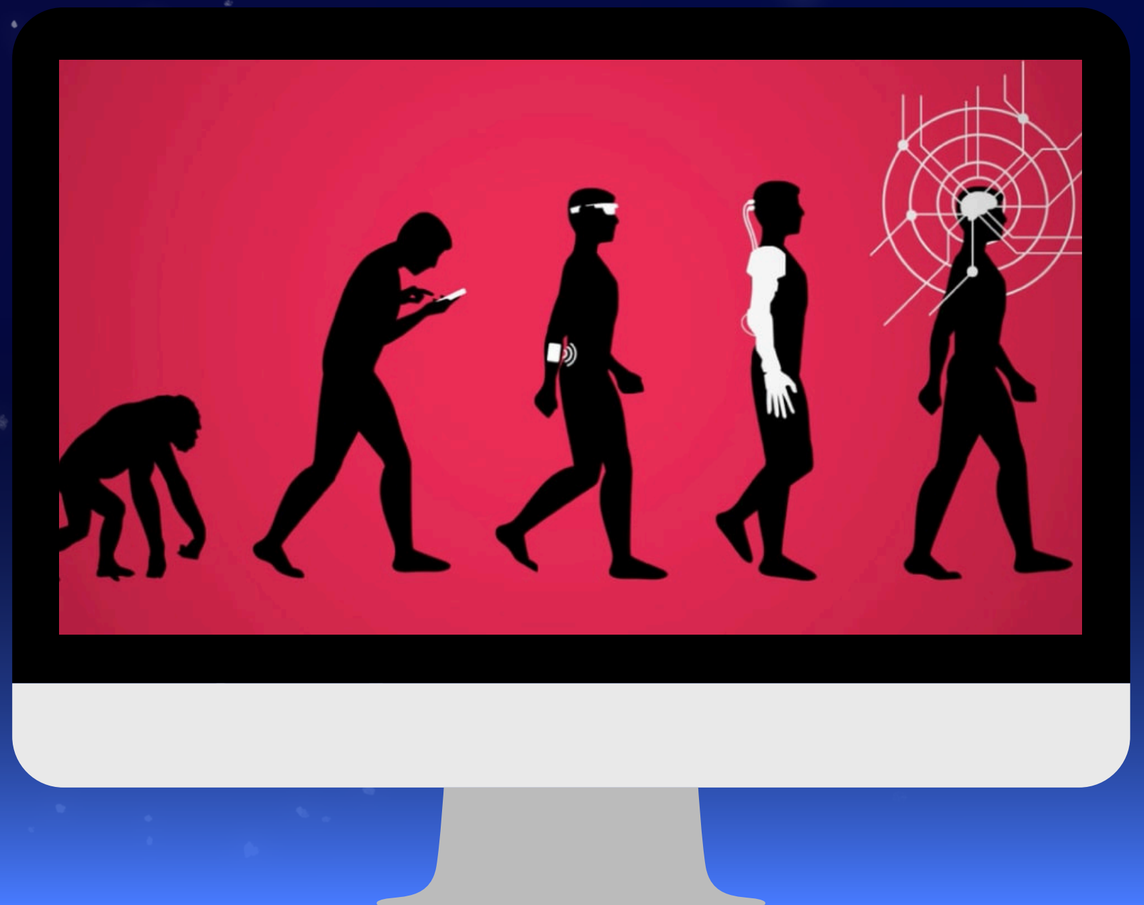
Outline

- Definition
- Origin and Evolution
- Importance
- How it works
- Basic Framework of EC
- Types
- Real-World application
- Pros & Cons

What is Evolutionary Computation?

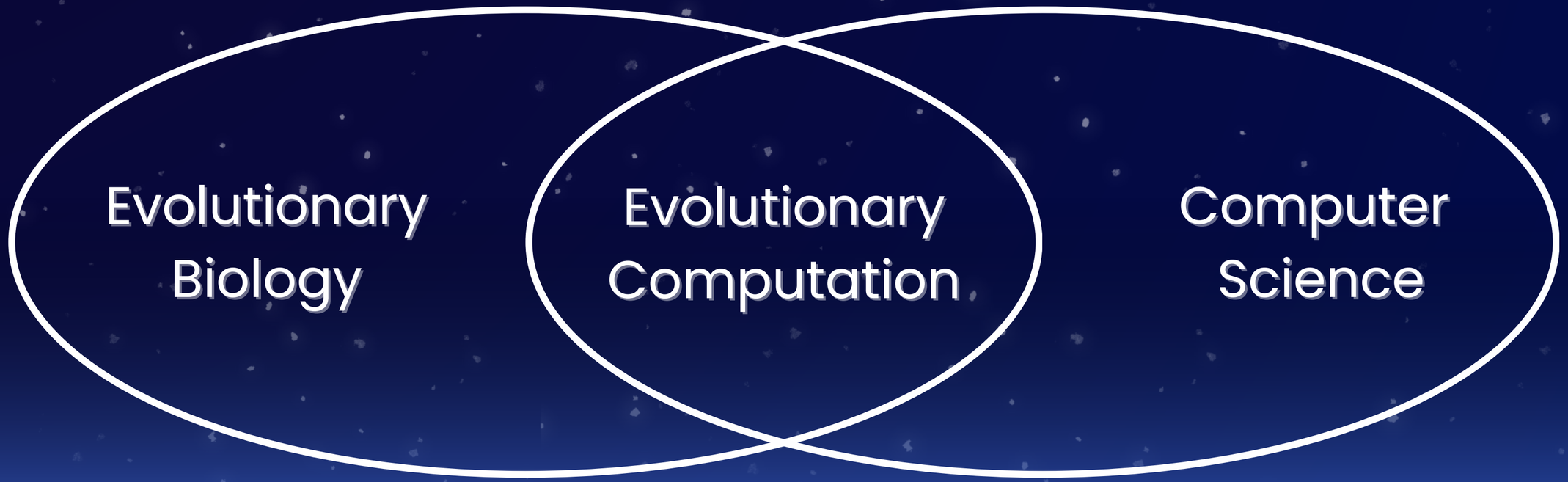
MEANING

A type of algorithm inspired by the process of natural selection in biology. It uses techniques to solve complex problems.



What is Evolutionary Computation?

Relationship



What is Evolutionary Computation?

Evolutionary Algorithms (EAs) are methods inspired by biological evolution.

Initialization

potential solutions

stop condition is met, for example a good solution found

good solution not found continue cycle

how good a solution is

Evaluation

Stop Criteria

choosing the best solution

Selection

Combining parts of two solutions to create new ones

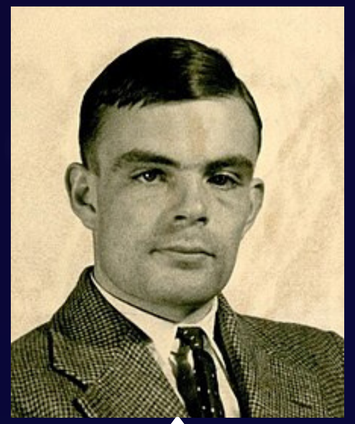
Reinsertion

Crossover

modifying a solution & introducing variation

History of Evolutionary Computation

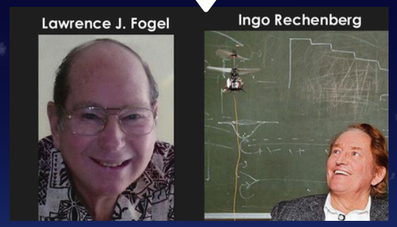
1950'S



Alan Turing suggests “genetical search” as a problem-solving method.

1960'S-70'S

- Genetic Algorithms by John Holland.
- Evolutionary Strategies by Ingo Rechenberg & Hans-Paul Schwefel.
- Evolutionary Programming by Lawrence Fogel.



1980'S

Genetic Programming (GP) introduced by John Koza.

1990'S

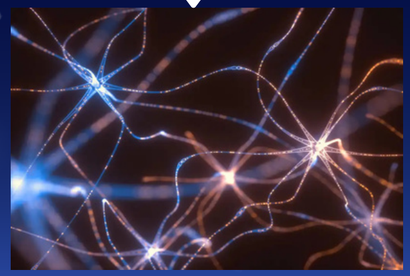
Evolutionary computation gains traction in artificial intelligence, optimization, and robotics.

2000'S

Evolutionary computation applied in engineering, bioinformatics, and hybrid methods.

2010'S - NOW

Integration with deep learning for neural architecture search and optimization.



The Significance of Evolutionary Computation in AI

It is important in AI because it helps systems learn & adapt, finding better solutions to complex problems & optimizing processes.

FINANCE

Evolutionary algorithms improve trading strategies by adapting to market changes, helping manage risk & make better investment decisions.

HEALTHCARE

These algorithms help advance drug discovery & create personalized treatment plans, tailored to each person's unique genetics for best results.

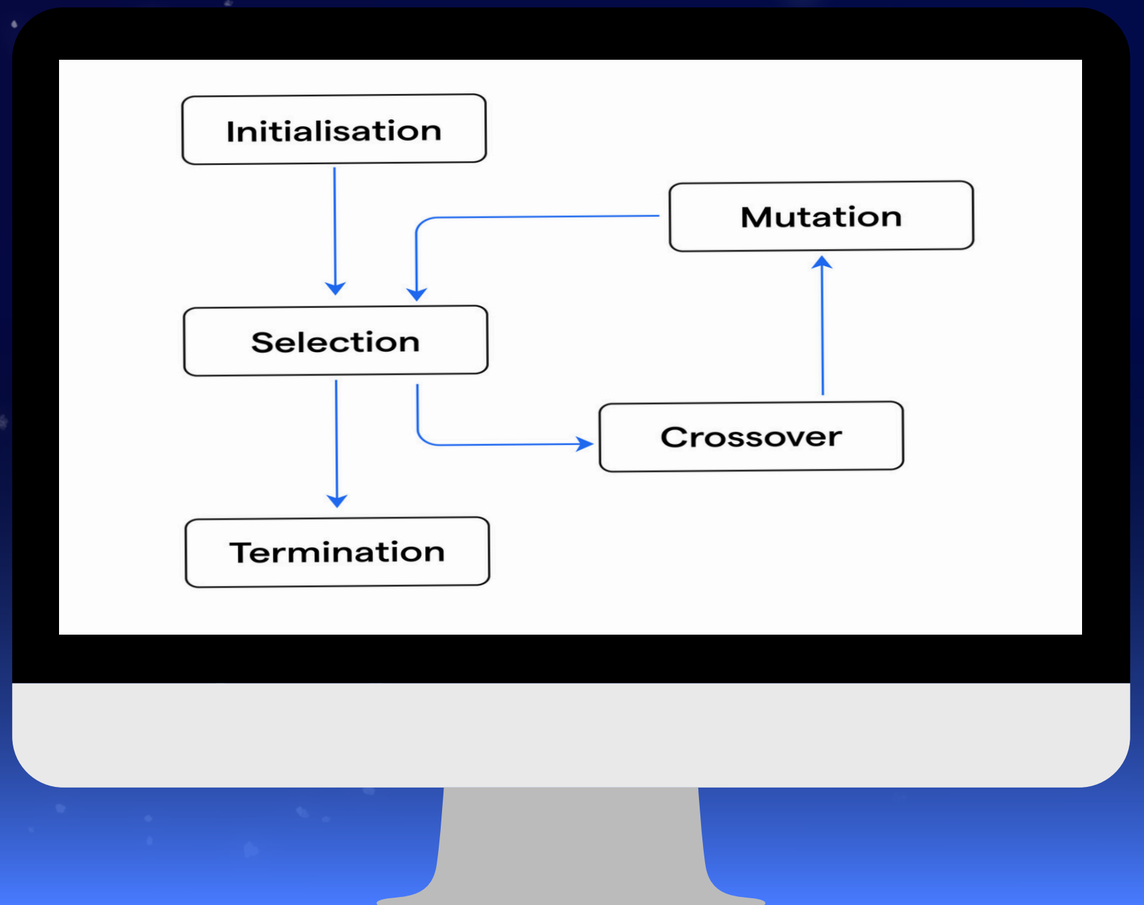
ROBOTICS

Evolutionary algorithms can help robots learn and adapt to new environments, making them more autonomous and efficient.

How evolutionary computation works

EXPLANATION

A method that creates and improves solutions by imitating natural evolution. It starts with a group of possible solutions and uses processes like selection, mixing, and small changes (mutation) to make them better with each step, or generation. This way, solutions improve slowly over time.



HOW IT WORKS

1 DEFINE THE PROBLEM AND OBJECTIVES

- **Outlining the problem that evolutionary computation will address.**
- **Establish the specific objectives and constraints that the algorithm should optimize for.**

Cookie Example

Define the Problem and Objectives

Problem:

We want to make the best-tasting cookie

Objective: Find a mix of the best ingredients (sugar, chocolate chips, & flour) to make the most delicious cookie

Constraints: The cookie should be baked with ingredients in the right proportions (not too much/too little of each ingredient).

2 DESIGN THE GENETIC REPRESENTATION

- **Define a suitable encoding scheme to represent potential solutions as individuals within a population.**
- **Determine the genetic operators, such as crossover and mutation, that will enable the exploration of solution space.**

Cookie Example

Design the Genetic Representation

Encoding Scheme: Each recipe will have a "list" of ingredients, with different amounts of sugar, chocolate chips, and flour.

Genetic Operators:

Crossover: Mix two different recipes to make a new recipe.

Mutation: Randomly change one ingredient (e.g. add more chocolate or reduce sugar).

3 INITIALIZATION & POPULATION GENERATION

- **Initialize a population of potential solutions based on the defined genetic representation.**
- **Ensure diversity within the initial population to facilitate robust exploration of the solution space.**

Cookie Example
Initialization and Population Generation

Population: Start with a few random recipes. Each recipe is different, like one has lots of sugar, and another has more chocolate chips.

Diversity: Each initial recipe has different proportions of ingredients, so we have a mix of flavors to start with.

4 ITERATIVE EVOLUTIONARY PROCESS

- **Implement the evolutionary cycle involving selection, recombination, and mutation operations.**
- **Iteratively evaluate, select, and evolve the population towards achieving optimal solutions.**

Cookie Example:
Iterative Evolutionary Process

Evaluation: Taste each recipe to see how yummy it is (like giving it a score).

Selection: Choose the best-tasting recipes to keep and make new recipes from.

Recombination and Mutation:

1. Mix two good recipes together (crossover) to create a new one. Change one ingredient a little in some recipes (mutation) to see if it tastes better.
2. Repeat: Continue testing, selecting, mixing, and changing until the recipes get yummier.

5 TERMINATION & SOLUTION EXTRACTION

- **Define the termination criteria, such as reaching a predefined fitness threshold or a maximum number of generations.**
- **Extract the best solutions generated through the evolutionary process for further evaluation or utilization.**

Cookie Example

Termination and Solution Extraction

Termination:

Stop when we find a cookie recipe that’s super yummy or after trying a set number of recipes.

Best Recipe:

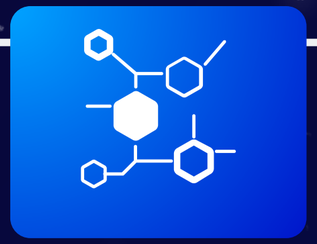
The yummiest recipe at the end is our winner!

MAIN TYPES OF EC

4 Main Types of Evolutionary Computation

GENETIC ALGORITHMS (GA)

an optimization method that mimics natural selection to find the best solution by evolving a population of potential solutions over generations.



GENETIC PROGRAMMING(GP)

Creates and evolves computer programs to solve problems by mimicking natural selection

EVOLUTIONARY PROGRAMMING (EP)

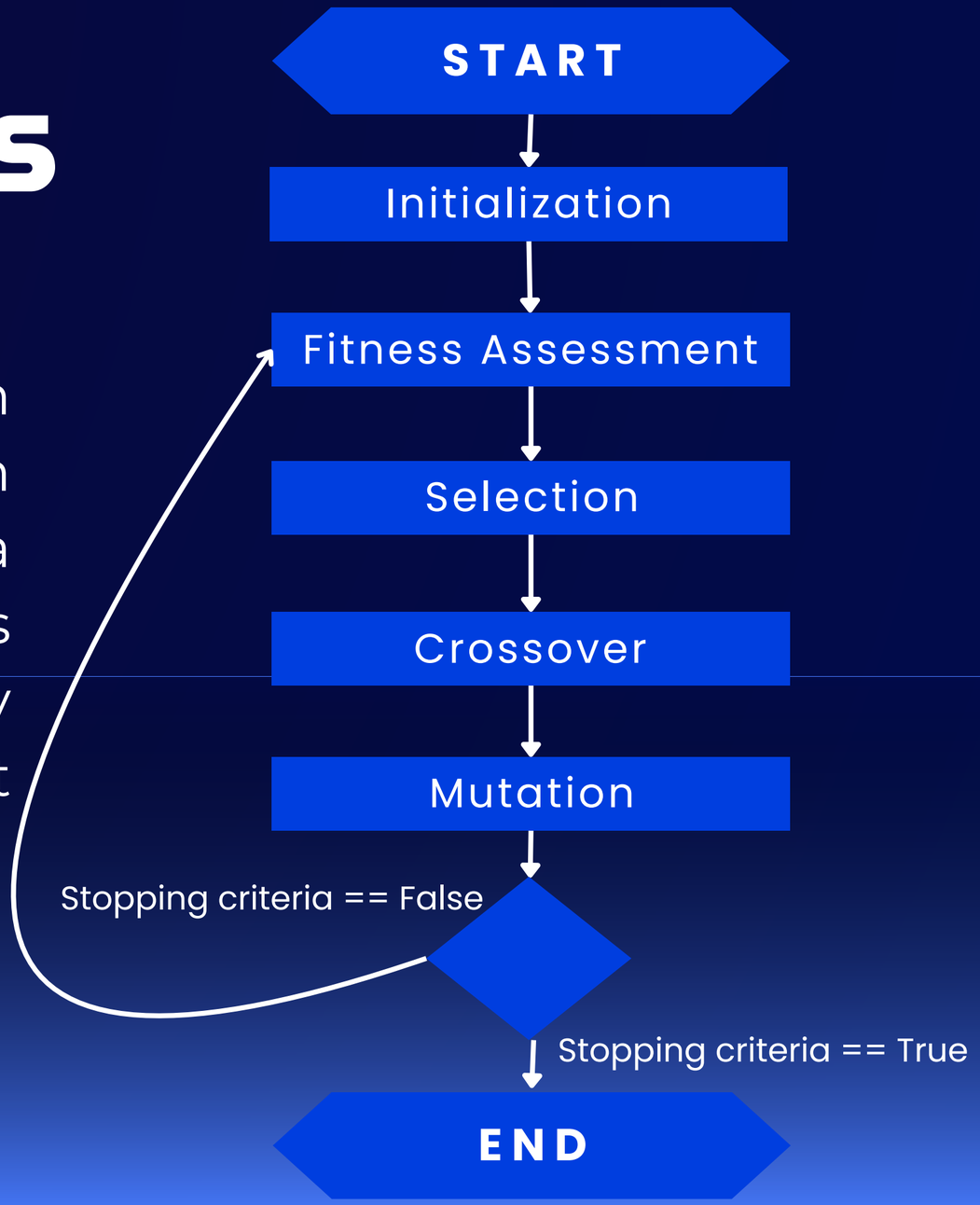
The programs that need to be optimized have a fixed structure, while the numeric parameters can evolve.

EVOLUTIONARY STRATEGIES (ES)

Evolution strategies are optimization algorithms inspired by natural evolution, using mutation, selection, and recombination to improve solutions iteratively.

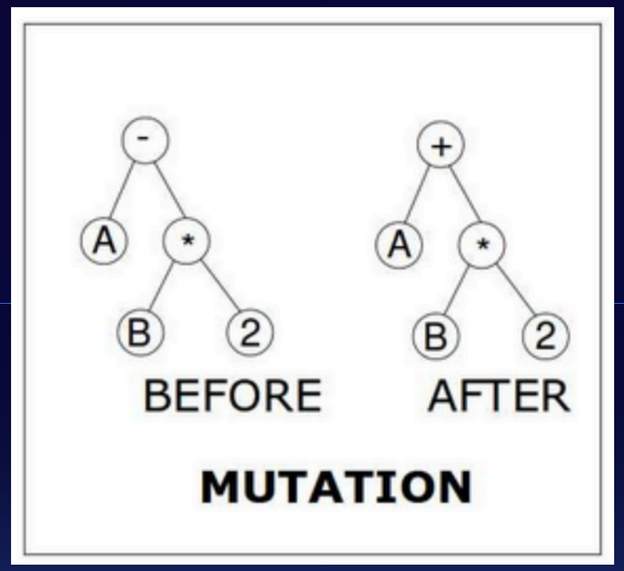
Genetic Algorithms

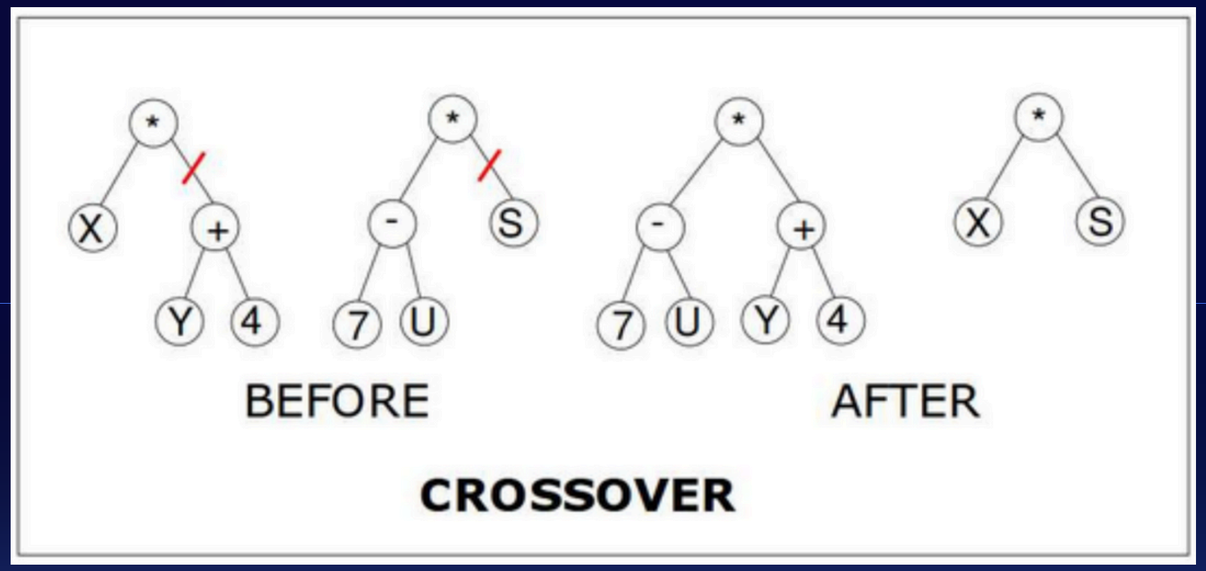
An optimization technique where a population of candidate solutions evolves through crossover, mutation, and selection based on a fitness function. This iterative process refines solutions over generations, gradually converging toward the optimal or most efficient answer.



Genetic Programming

(GP) is a method that evolves computer programs to solve problems. Starting with random programs, GP improves them through random changes and selection based on quality. Over time, these programs evolve to solve specific tasks automatically.



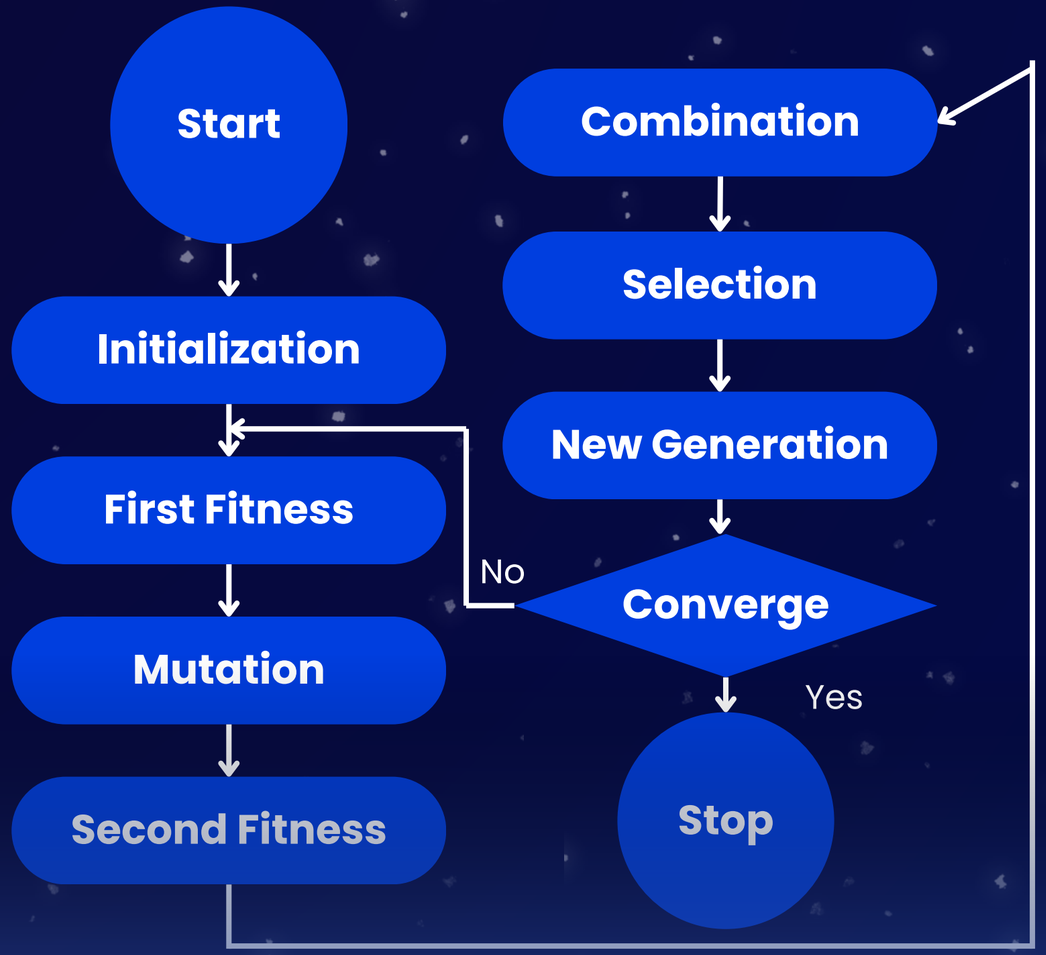


Random Change:
The **“ - ”** operator at the top is replaced with **“ + ”**, creating a new variant of the expression.

Combining parts from two "Parent" structures to create new "Offspring" :
Here, a subtree from the first parent (rooted at +) is swapped with a subtree from the second parent (rooted at S). This results in two new expressions in the "After" section.

Evolutionary Programming

Evolutionary Programming (EP) is a type of evolutionary computation that focuses on optimizing solutions by simulating the evolution of finite state machines (FSMs) or other representations of solutions.



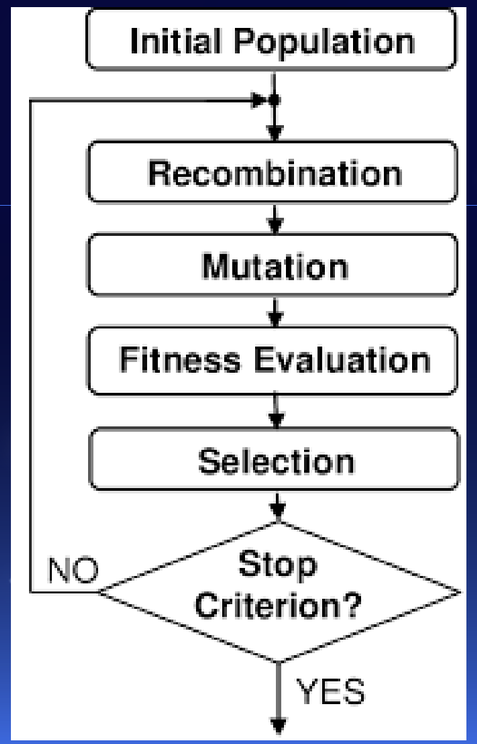
Evolutionary Strategies

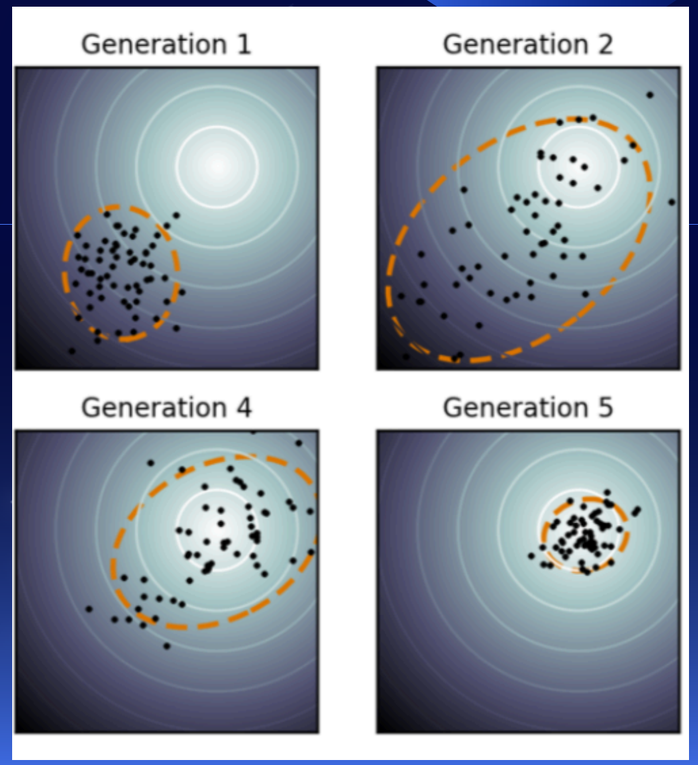
DEFINITION

Evolutionary Strategies is a type of algorithm inspired by natural evolution. It's used to find the best solution to a problem by mimicking how living organisms evolve over time.

HOW IT WORKS

- Start with random solutions for a problem
- Make small changes to create "offspring" solutions.
- Select the best solutions to form the next "generation."
- Repeat to keep improving toward the best answer.



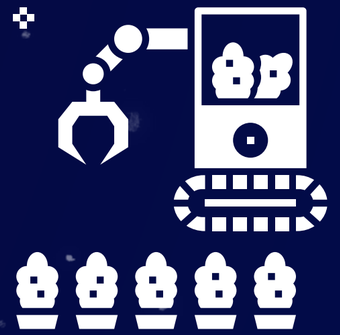


REAL WORLD APPLICATIONS

Real-world applications of Evolutionary Computation in AI

Evolving Neural networks for image recognition

Genetic algorithms in financial modeling



Evolutionary strategies in robotics

Example 1 : Evolving neural networks for image recognition

Application → Computer Vision

Goal → Uses evolutionary computation to optimize neural network architectures for image recognition.

Process

Evolves network structure and connection weights. → Iteratively improves architecture for better performance.

Example 2 : Genetic algorithms in financial modeling

Application → Financial modeling

Goal → Uses genetic algorithms to optimize financial strategies and predictions.

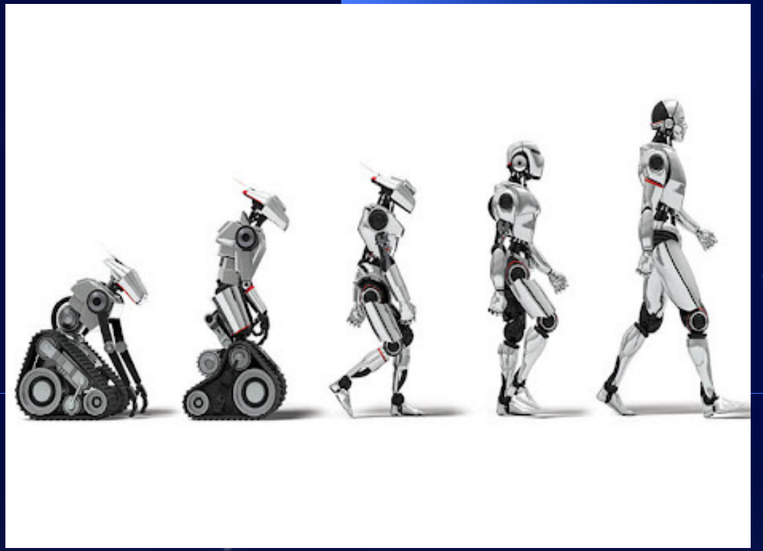
Process

Evolves parameters and trading rules. → Selects best-performing strategies over iterations.

Example 3: Evolutionary strategies in robotics

Application → Robotic Locomotion and Control

Goal → Use evolutionary strategies to improve how robots move and adapt to their environment.



Process

Start with various movement parameters (like speed, balance, joint angles) for the robot. → Use evolutionary strategies to make gradual adjustments to these parameters. → Test each version of the robot in a simulation to see how well it moves.

ADVNATAQES

&

DISADVANTAQES

CHALLENGES of EC

Reason	Explanation
Computational Complexity	Some evolutionary algorithms may exhibit high computational demands, especially for problems with large solution spaces.
Premature Convergence	The exploration-exploitation balance in evolutionary algorithms can lead to premature convergence to suboptimal solutions in certain scenarios.
Parameter Sensitivity	Fine-tuning the parameters of evolutionary computation algorithms for optimal performance can require domain-specific knowledge and extensive experimentation.

ADVANTAGES of EC

Reason	Explanation
Versatility	EC Techniques can be applied to a wide range of problem domains, showcasing adaptability and robustness.
Global Optimization	By exploring solution spaces using population-based methods, evolutionary computation can identify global optima for complex problems.
Adaptability	The iterative nature of evolutionary algorithms enables them to adapt to dynamic environments and changing objectives.

Thank You